

Bluetooth PA System: Project Manual

Mobile Microphone to ESP32 Speaker

Project Guide

November 27, 2025

1 Project Overview

This project creates a wireless Public Address (PA) system. It allows a user to speak into their mobile phone's microphone, transmit the audio via Bluetooth A2DP, and have it played out loud through a speaker connected to an ESP32 microcontroller.

How it Works

1. **Input:** The Android app captures voice data from the microphone.
2. **Transmission:** The app routes the audio to the phone's "Music Stream," which the Android OS automatically transmits via Bluetooth.
3. **Reception:** The ESP32 receives the digital audio stream.
4. **Output:** An I2S Amplifier (MAX98357A) converts the digital signal to analog power to drive the loudspeaker.

2 Bill of Materials

Hardware

- **ESP32 Development Board:** Standard ESP32-WROOM-32 or DevKit V1.
- **MAX98357A I2S Amplifier Module:** A low-cost, high-efficiency Class-D amplifier breakout board.
- **Loudspeaker:** 4Ω or 8Ω impedance, rated for 3 Watts.
- **Power Supply:** A high-quality Micro-USB cable and a 5V/2A power adapter (or power bank).
- **Jumper Wires:** Female-to-Female or Male-to-Female depending on your headers.

Software

- **Arduino IDE:** For programming the ESP32.
- **ESP32-A2DP Library:** Created by Phil Schatzmann (available via Arduino Library Manager).
- **Android Studio:** For creating the mobile microphone app.

3 Hardware Assembly

Wiring Diagram

Connect the ESP32 to the MAX98357A amplifier using the following pin mapping.

MAX98357A Pin	ESP32 Pin	Function
LRC	GPIO 25	Left/Right Clock (Word Select)
BCLK	GPIO 26	Bit Clock
DIN	GPIO 22	Data Input
VIN	5V (or VIN)	Power Supply
GND	GND	Ground

Table 1: I2S Wiring Connections

Speaker Connection

Connect the two wires from your loudspeaker to the screw terminal block on the MAX98357A. Polarity (+/-) is not critical for a single speaker setup, but ensure the wires are screwed in tightly.

4 ESP32 Firmware Setup

1. Library Installation

1. Open Arduino IDE.
2. Go to **Sketch > Include Library > Manage Libraries**.
3. Search for **ESP32-A2DP**.
4. Install the version by *Phil Schatzmann*.

2. Low Latency Configuration

To minimize the delay between speaking and hearing the audio, use the following configuration in your Arduino sketch setup:

```
// Standard Setup
BluetoothA2DPSink a2dp_sink;

void setup() {
  // Configure I2S Pins
  i2s_pin_config_t my_pin_config = {
    .bck_io_num = 26,
    .ws_io_num = 25,
    .data_out_num = 22,
    .data_in_num = I2S_PIN_NO_CHANGE
  };
  a2dp_sink.set_pin_config(my_pin_config);
}
```

```
// Start Bluetooth
a2dp_sink.start("ESP32-PA-System");
}
```

5 Mobile App Testing Logic

You do not need to write a complex Bluetooth stack. The Android app simply routes microphone audio to the system music stream.

Manual Testing Steps

1. Power on the ESP32. Wait for it to initialize.
2. Open your phone's **Settings > Bluetooth**.
3. Pair with the device named **"ESP32-PA-System"**.
4. Ensure "Media Audio" is enabled in the Bluetooth device settings.
5. Open your custom app (or use a third-party "Microphone to Speaker" app for testing).
6. Speak into the phone. The audio should emerge from the ESP32 speaker.

6 Troubleshooting & Safety

Common Issues

- **Choppy Audio:** This is usually caused by insufficient power. WiFi and Bluetooth radio bursts require significant current. Ensure you are using a 2A power supply, not a weak computer USB port.
- **Loud Screeching (Feedback):** This occurs if the phone is too close to the speaker. Move the phone away or lower the volume.
- **High Latency (Delay):** Bluetooth A2DP inherently has 100-200ms of latency. To reduce this further, minimize buffer sizes in both the Android 'AudioRecord' setup and the ESP32 I2S config.

Safety Warning

Never connect a loudspeaker directly to the ESP32 GPIO pins. The speaker draws high current (approx 800mA) which exceeds the ESP32 pin limit (40mA), likely destroying the pin or the board. Always use the MAX98357A amplifier.